

Smart Plant Carer · Full Curriculum

Single session (2.5 hrs) · 9-13 yrs · Beginner → Intermediate

Full facilitator curriculum **PREMIUM**

Overview

Build a self-watering planter that senses dry soil and pumps water to a real seedling all by itself.

DETAIL

Track	Arduino Robotics & Electronics
Format	Single session (2.5 hrs)
Ages	9-13 yrs
Level	Beginner → Intermediate
Price	from KES 5,000
Standalone	Yes
Prerequisites	None — standalone

Course Overview

Smart Plant Carer is a single 2.5-hour Arduino build in which every student wires, codes and takes home a working self-watering planter. Students start by learning how a soil-moisture sensor "sees" wet and dry soil, move on to controlling a real water pump through a relay so the system waters only when needed, and finish by assembling the electronics around a real potted seedling and running a full, hands-off watering cycle.

The session is built as one continuous engineering story: sense → decide → act. In Part 1 students calibrate a sensor and read raw numbers on the Serial Monitor. In Part 2 they add logic and a relay-driven pump so the Arduino makes its own watering decisions. In Part 3 they become plumbers and product designers, fitting

tubing, a reservoir and a pump around a living seedling so the whole system runs unattended.

By the end of the session every student takes home a complete, working self-watering planter kit with their own seedling planted in it — a device they built, coded and calibrated themselves, that keeps a real plant alive after they leave.

Learning Outcomes

- Wire and read an analog sensor (capacitive soil moisture sensor) into an Arduino and interpret raw ADC values (0–1023) on the Serial Monitor.
- Calibrate a sensor by recording real-world "wet" and "dry" reference values and convert raw readings into a meaningful 0–100% scale using `map()` and `constrain()`.
- Understand how a relay module lets a low-power microcontroller safely switch a higher-power/higher-current device (a water pump) on a separate power supply.
- Write conditional logic that makes an autonomous decision (if moisture < threshold, then water).
- Understand and implement a simple non-blocking state machine using `millis()` to sequence "check → water → cooldown" behaviour without `delay()` freezing the whole program.
- Assemble a real, waterproofed mechanical system (tubing, reservoir, pump, pot) around a living plant safely and correctly.
- Test, debug and calibrate a finished physical-computing product end-to-end and explain how it works to an audience.

Materials Master List

ITEM	SPEC	QTY PER STUDENT	NOTES
Microcontroller board	Arduino Uno R3 (or compatible clone, e.g. from Pixel Electronics/ Nerokas)	1	Reused across all 3 parts; goes home in the kit
USB cable	USB-A to USB-B	1	For programming; laptop can double as power source during class
Breadboard	830 tie-point half+ size	1	For sensor/relay/LED wiring

Jumper wires	Male-to-male, assorted lengths, pack of 10+	1 pack	
Jumper wires	Male-to-female, pack of 6+	1 pack	For connecting sensor/relay headers to breadboard
Soil moisture sensor	Capacitive soil moisture sensor v1.2, analog output, 3.3-5V	1	Capacitive preferred over resistive — doesn't corrode in wet soil
Relay module	5V single-channel relay module with opto-isolation (e.g. Songle SRD-05VDC-SL-C board)	1	Switches pump circuit; controlled by Arduino digital pin
Water pump	Mini submersible DC water pump, 3-6V, low current (~130-220 L/h rated)	1	Runs off separate battery pack, not the Arduino
Silicone tubing	Food-grade silicone tubing, 2-4 mm ID, ~50 cm length	1 length	Connects pump outlet to soil
Pump power supply	4×AA battery holder with switch + 4 AA batteries (6V)	1	Powers pump through the relay; kept separate from Arduino's 5V rail
Arduino power supply (take-home)	9V battery + barrel jack battery clip, or 5V/1A USB wall adapter	1	So the planter can run unattended at home without a laptop
Status LED	5mm LED, any colour	1	Lights up while pump is running
Resistor	220Ω, 1/4W	1	Current-limiting resistor for LED
Seedling	Small potted seedling (e.g. bean, coriander, spinach or basil)	1	Take-home living component
Planting pot	Small plastic pot with drainage holes, ~10-12 cm diameter	1	Houses the seedling and sensor
Potting soil	General potting mix, small bag	~1 cup per student	Fill pot around seedling
Water reservoir	500 ml-1 L plastic bottle or small food container with lid	1	Holds water supply; pump sits inside
Hot glue gun + sticks	Mini hot glue gun, glue sticks	1 per 4 students (shared)	For securing tubing and sealing reservoir lid
small amount			

Waterproofing tape/ sealant	Silicone sealant strip or waterproof tape		Seal the hole where tubing passes through reservoir lid
Cable ties	Small, 100mm	3-4	Tidy wiring, secure tubing
Craft knife / scissors	Standard	1 per 4 students (shared)	Cutting tubing and holes in bottle caps
Awl or small screwdriver	For piercing holes in plastic bottle caps	1 per 4 students (shared)	Facilitator-supervised only
Spill tray	Plastic basin or tray, ~30x20cm	1 per student	Catches drips during testing
Paper towels / cloth	Roll	shared	Spill cleanup
Permanent marker	Fine tip	1 per 4 students (shared)	Marking cut lines and calibration notes
Laptop with Arduino IDE installed	Arduino IDE 1.8.x or 2.x, Serial Monitor enabled	1 per student/ pair	Pre-install drivers before session

Session Plans

Part 1 — Sense the Soil

Objectives

- Correctly wire a capacitive soil moisture sensor to the Arduino.
- Read and interpret raw analog values (0-1023) on the Serial Monitor.
- Record real "wet" and "dry" calibration values for their own sensor.

Timing (45 minutes)

- 0:00-0:10 — Welcome, safety briefing (water + electronics rules), show finished planter as the goal
- 0:10-0:20 — Explain how a capacitive soil sensor works (measures capacitance change with moisture, safer than resistive probes)
- 0:20-0:30 — Guided wiring of sensor to breadboard/Arduino
- 0:30-0:40 — Upload sketch, open Serial Monitor, calibrate: dip sensor tip in a cup of water, then hold in dry air/dry soil, record both numbers
- 0:40-0:45 — Quick check-in: everyone has their two calibration numbers written down

Step-by-step

1. Open with a question: "How would a plant tell you it's thirsty if it could talk?"
Lead into: today the sensor is its voice.
2. Hand out breadboards, Arduinos and sensors. Demonstrate the 3-pin header on the sensor: VCC, GND, AOUT.
3. Guide wiring one connection at a time, checking each table before moving on (see wiring list below).
4. Live-type the Part 1 sketch together on the projector, explaining `analogRead()` and `Serial.println()` line by line.
5. Upload and open Serial Monitor (set baud rate to 9600 — point this out explicitly, it's a common miss).
6. Have students dip only the sensor's metal paddle (not the PCB/header end) into a cup of water — read and record the number as WET_VALUE.
7. Dry the sensor with a paper towel, hold in open air (or push into dry soil) — record as DRY_VALUE.
8. Write both numbers on a sticky note stuck to their breadboard — they'll need them in Part 2.

Build detail

Materials: Arduino Uno, USB cable, breadboard, capacitive soil moisture sensor, 3× male-to-female jumper wires, laptop with Arduino IDE, small cup of water, paper towel.

Wiring (component → pin):

- Soil sensor VCC → Arduino 5V
- Soil sensor GND → Arduino GND
- Soil sensor AOUT → Arduino A0

```
// Session 1: Sense the Soil
// Reads the analog output of the capacitive soil moisture sensor
// and prints it to the Serial Monitor so we can find the WET and DRY numbers.

const int soilPin = A0;    // Soil moisture sensor AOUT wire

void setup() {
  Serial.begin(9600);       // Open Serial Monitor at 9600 baud
  Serial.println("Soil Moisture Calibration");
  Serial.println("Move the sensor between air, dry soil and water and watch the numbers.");
}
```

```
void loop() {
  int reading = analogRead(soilPin);  // 0 (very wet) - 1023 (very dry), roughly
  Serial.print("Raw sensor value: ");
  Serial.println(reading);
  delay(500);                          // update twice a second
}
```

Checks for understanding

- "Which pin does the sensor's data wire (AOUT) connect to, and why not a digital pin?"
- "If your dry reading is 620 and your wet reading is 280, which number means MORE water?"
- "Why do we get a slightly different reading each time even if the sensor hasn't moved?" (noise — expect small fluctuation)

MISTAKE	FIX
Serial Monitor shows garbled characters	Check baud rate dropdown in Serial Monitor is set to 9600, matching <code>Serial.begin(9600)</code>
Reading is always 1023 or 0 and never changes	Check AOUT wire is actually in A0, not A1 or a digital pin; check VCC/GND aren't swapped
Submerging the whole sensor including the PCB end	Only the coated metal paddle should touch water/soil — the header end must stay dry

Part 2 — Pump on Demand

Objectives

- Wire a relay module to safely switch a water pump on a separate power supply.
- Convert raw sensor readings into a 0–100% moisture value using calibration data.
- Write conditional logic so the pump only runs when soil is below a chosen threshold.

Timing (50 minutes)

- 0:00–0:10 — Explain relays: why we can't plug a pump straight into an Arduino pin (current limits), how a relay acts as an electronic switch
- 0:10–0:25 — Guided wiring of relay, pump, battery pack and status LED
- 0:25–0:35 — Live-code Part 2 sketch together, entering their own DRY_VALUE/WET_VALUE from Part 1

- 0:35-0:45 — Test: dip sensor in water and in dry soil, watch pump switch on/off; adjust `MOISTURE_THRESHOLD` together
- 0:45-0:50 — Troubleshooting circulation, quick group check-in

Step-by-step

1. Ask: "The Arduino pin can only supply a tiny trickle of current — how do we control something as thirsty as a pump?" Introduce the relay as an electrically-controlled switch.
2. Show the relay board: COM/NO/NC screw terminals for the pump's high-power side, and IN/VCC/GND pins for the Arduino's low-power control side. Stress these are two separate circuits.
3. Guide wiring carefully — this is the highest-risk step for miswiring. Check every group's relay wiring before power-on.
4. Wire the battery pack in series with the pump through the relay's COM and NO terminals (relay open = pump off, closed = pump on).
5. Add the status LED with its resistor on a spare digital pin so students get visual + serial feedback.
6. Type the sketch together, pausing to explain `map()` turning raw numbers into a 0-100% scale, and that most relay boards are LOW-triggered (a quirk worth explicitly calling out).
7. Each student edits `DRY_VALUE` and `WET_VALUE` to match their own Part 1 numbers before uploading.
8. Test live: touch sensor to wet soil/water → LED off, pump off. Move to dry paper towel/dry soil → LED on, pump runs. Adjust `MOISTURE_THRESHOLD` if it triggers too early or late.

Build detail

Materials: everything from Part 1, plus relay module, mini submersible pump, silicone tubing offcut, 4×AA battery pack + batteries, LED, 220Ω resistor, small cup of water, spill tray.

Wiring (component → pin):

- Soil sensor VCC → 5V, GND → GND, AOUT → A0 (unchanged from Part 1)
- Relay VCC → Arduino 5V
- Relay GND → Arduino GND
- Relay IN → Arduino pin 7

- Battery pack (+) → Relay COM
- Relay NO → Pump (+)
- Pump (-) → Battery pack (-)
- LED long leg (anode) → 220Ω resistor → Arduino pin 8; LED short leg (cathode) → GND

```
// Session 2: Pump on Demand
// Turns the water pump ON via a relay when soil moisture is below a threshold.

const int soilPin = A0;
const int relayPin = 7;
const int ledPin = 8;

// Calibration values from Session 1 - EDIT THESE for your own sensor!
const int DRY_VALUE = 620; // raw reading in completely dry soil/air
const int WET_VALUE = 280; // raw reading in a cup of water

const int MOISTURE_THRESHOLD = 40; // % - pump turns on below this

void setup() {
  Serial.begin(9600);
  // Set the pin HIGH *before* making it an output. If we enable the output
  // first it drives LOW for an instant, which on a LOW-triggered relay kicks
  // the pump on every reset.
  digitalWrite(relayPin, HIGH); // Most relay modules are LOW-triggered; HIGH = pump OFF
  pinMode(relayPin, OUTPUT);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
}

void loop() {
  int raw = analogRead(soilPin);

  // Convert raw reading into a 0-100% moisture value
  int moisture = map(raw, DRY_VALUE, WET_VALUE, 0, 100);
  moisture = constrain(moisture, 0, 100); // keep it within 0-100 even if raw drifts

  Serial.print("Raw: ");
  Serial.print(raw);
  Serial.print("  Moisture: ");
  Serial.print(moisture);
  Serial.println("");

  if (moisture < MOISTURE_THRESHOLD) {
    digitalWrite(relayPin, LOW); // LOW = relay energised = pump ON
    digitalWrite(ledPin, HIGH);
    Serial.println("Soil is DRY -> Pump ON");
  } else {
    digitalWrite(relayPin, HIGH); // pump OFF
    digitalWrite(ledPin, LOW);
    Serial.println("Soil is OK -> Pump OFF");
  }

  delay(1000);
}
```

Checks for understanding

- "Why does the pump need its own battery pack instead of running off the Arduino's 5V pin?"
- "If `MOISTURE_THRESHOLD` is 40 and the reading is 25%, does the pump turn on or off? Why?"
- "What does `digitalWrite(relayPin, LOW)` actually do on this relay board?"

MISTAKE	FIX
Pump runs constantly regardless of moisture	Check relay logic — most modules are LOW-triggered; confirm wiring matches IN pin used in code
Relay clicks but pump doesn't run	Check battery pack polarity and that pump wires are on COM/NO, not on the Arduino-side header
Moisture % reads negative or over 100	DRY_VALUE and WET_VALUE weren't updated from their own Part 1 readings, or <code>constrain()</code> line was removed
Arduino resets/browns out when pump starts	Confirm pump is powered from the separate battery pack, not sharing the Arduino's 5V/USB power

Part 3 — Build the Planter

Objectives

- Physically assemble pump, tubing, reservoir and sensor around a real potted seedling.
- Upgrade the code to a full automatic watering cycle using timed "watering" and "cooldown" phases.
- Run and demonstrate an end-to-end automatic watering system.

Timing (55 minutes)

- 0:00–0:10 — Show the finished planter assembly (photo/sample), explain the plumbing plan: reservoir → pump → tubing → soil
- 0:10–0:15 — Safety note on hot glue gun and piercing bottle caps; facilitator demonstrates first
- 0:15–0:35 — Guided physical assembly: pot the seedling, position sensor, feed tubing, seal reservoir lid, mount pump inside reservoir
- 0:35–0:45 — Upgrade code to the full state-machine sketch; explain "check → water → cooldown" cycle

- 0:45-0:50 — Full system test: add water to reservoir, dry out soil slightly, watch full automatic cycle run at least once
- 0:50-0:55 — Clean-up, pack take-home kit, transition into showcase

Step-by-step

1. Walk students through the mechanical plan on the board: pump sits submerged in the reservoir, tubing runs from pump outlet, through a hole in the reservoir lid, into the soil near the seedling's roots.
2. Facilitator demonstrates using the awl to pierce two small holes in the reservoir lid (one for tubing, one for the pump's power wires) — students should not do this step unsupervised for younger groups; facilitator or a supervised pairing does the piercing.
3. Students plant their seedling into the pot with fresh potting soil, leaving room to push the soil sensor paddle and the end of the tubing into the soil, a few centimetres apart.
4. Feed the tubing through the lid hole, secure with a small bead of hot glue or waterproof tape so no water leaks around it.
5. Place the pump inside the reservoir, feed its wires out through the second hole, seal similarly.
6. Fill the reservoir with water, position it beside the pot, secure loose wiring with cable ties, and place everything on the spill tray.
7. As a class, upgrade the Part 2 sketch to the Part 3 state-machine sketch — explain that instead of switching the pump every second, the system now runs a short water burst then waits ("cooldown") so water has time to soak into the soil before checking again.
8. Test the whole system: if soil is already moist, get students to gently dry the top layer or lift the sensor briefly to trigger a watering event and observe the pump run for exactly 3 seconds before switching off and entering cooldown.
9. Confirm each planter completes at least one full check→water→cooldown cycle before ending the session.

Build detail

Materials: everything from Part 2, plus potted seedling, potting soil, planting pot, water reservoir container with lid, hot glue gun, waterproof sealant/tape, cable ties, awl, scissors/craft knife, spill tray, water for filling reservoir.

Wiring/assembly (component → location):

- Sensor paddle → pushed into soil, ~2-3 cm deep, near seedling roots
- Sensor wires → routed up out of the pot to breadboard/Arduino (unchanged pins from Part 1/2)
- Pump → fully submerged inside water reservoir
- Pump outlet → silicone tubing → fed through sealed hole in reservoir lid → positioned so tubing end sits just above/in the soil near the seedling
- Pump wires → fed through second sealed hole in reservoir lid → relay NO/battery pack (unchanged from Part 2)
- Relay, LED, Arduino wiring → unchanged from Part 2

```
// Session 3: Build the Planter - Full Automatic Watering Cycle
// Checks soil moisture regularly. If dry, runs the pump for a short burst,
// then waits (cooldown) before checking again so water has time to soak in.
// Uses millis() instead of delay() so the system never "freezes".

const int soilPin = A0;
const int relayPin = 7;
const int ledPin = 8;

// Calibration values from Session 1 - EDIT THESE for your own sensor!
const int DRY_VALUE = 620;
const int WET_VALUE = 280;
const int MOISTURE_THRESHOLD = 40; // % - below this, the plant needs water

const unsigned long WATER_DURATION = 3000; // pump runs for 3 seconds per watering
const unsigned long COOLDOWN_DURATION = 30000; // wait 30s after watering before re-checking
const unsigned long CHECK_INTERVAL = 2000; // check moisture every 2s while idle

enum State { CHECKING, WATERING, COOLDOWN };
State currentState = CHECKING;

unsigned long stateStartTime = 0;
unsigned long lastCheckTime = 0;

void setup() {
  Serial.begin(9600);
  digitalWrite(relayPin, HIGH); // pump off - set state before enabling output
  pinMode(relayPin, OUTPUT); // (otherwise the relay clicks on every reset)
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
  Serial.println("Smart Plant Carer - Automatic Watering Started");
}

int readMoisturePercent() {
  int raw = analogRead(soilPin);
  int moisture = map(raw, DRY_VALUE, WET_VALUE, 0, 100);
  return constrain(moisture, 0, 100);
}

void loop() {
```

```

unsigned long now = millis();

switch (currentState) {

  case CHECKING:
    // Only check moisture every CHECK_INTERVAL, not every single loop
    if (now - lastCheckTime >= CHECK_INTERVAL) {
      lastCheckTime = now;
      int moisture = readMoisturePercent();
      Serial.print("Moisture: ");
      Serial.print(moisture);
      Serial.println("%");

      if (moisture < MOISTURE_THRESHOLD) {
        Serial.println("Soil dry -> starting pump");
        digitalWrite(relayPin, LOW); // pump ON
        digitalWrite(ledPin, HIGH);
        stateStartTime = now;
        currentState = WATERING;
      }
    }
    break;

  case WATERING:
    // Stay in this state until WATER_DURATION has passed
    if (now - stateStartTime >= WATER_DURATION) {
      digitalWrite(relayPin, HIGH); // pump OFF
      digitalWrite(ledPin, LOW);
      Serial.println("Watering done -> cooling down");
      stateStartTime = now;
      currentState = COOLDOWN;
    }
    break;

  case COOLDOWN:
    // Wait for water to soak into the soil before checking again
    if (now - stateStartTime >= COOLDOWN_DURATION) {
      Serial.println("Cooldown finished -> checking soil again");
      currentState = CHECKING;
    }
    break;
}
}

```

Checks for understanding

- "Why doesn't the code use `delay()` for the watering and cooldown times?"
- "What would happen to the seedling if we removed the COOLDOWN state entirely?"
- "Where does the water physically go from reservoir to soil — trace the whole path."

MISTAKE

FIX

Water leaks around tubing/wire holes in the lid	Add a thicker bead of hot glue or wrap with waterproof tape; test by tilting the sealed reservoir over the spill tray before final placement
Tubing end sits in open air instead of soil	Reposition tubing tip so it's pressed a little into the soil surface near the seedling stem, not above the pot rim
Pump runs but no water reaches the soil	Check tubing isn't kinked, pump is fully submerged, and tubing is firmly pushed onto the pump outlet nozzle
System waters immediately even though soil looks moist	Re-check DRY_VALUE/WET_VALUE calibration — sensor may have shifted position or threshold is set too high

Assessment & Showcase

Assessment is formative and hands-on throughout the session rather than a written test. The facilitator circulates during each build phase and checks off, per student: correct sensor wiring and a recorded calibration pair (Part 1); correct relay/pump wiring with the pump responding to moisture changes (Part 2); and a fully assembled, leak-free planter that completes at least one full automatic check-water-cooldown cycle (Part 3).

"Done" for the session means: the seedling is potted with soil, sensor and tubing correctly placed; the reservoir is filled and sealed; the Arduino is running the Part 3 sketch with the student's own calibration values; and the pump has been observed switching on and off automatically at least once during class.

Close the session with a 15–20 minute showcase (can extend slightly past the 2.5 hours if the venue allows, or run within the final 10 minutes of Part 3's timing): each student places their planter on the table, briefly explains in their own words how their plant "knows" when to drink, and demonstrates the pump triggering (either naturally, or by lifting the sensor into dry air to force a "dry" reading live). Encourage students to state their own calibration numbers and threshold as their "plant's personality." Parents/guardians can be invited in for the last 10 minutes to watch demos.

Extension & Home Challenges

- **Easy:** Add a second LED (or change the existing one) that blinks slowly while the system is in COOLDOWN and stays solid while WATERING, so anyone can tell the planter's state at a glance without opening the Serial Monitor.

- **Medium:** Add a push-button "manual water" override wired to a digital pin, so pressing it runs the pump for one `WATER_DURATION` burst immediately, regardless of the current soil reading — useful for holidays or top-ups.
- **Hard:** Add a second soil sensor in a different spot in the pot and average the two readings before deciding to water, to make the system more resistant to one sensor being misplaced or reading unreliably. Bonus: log every watering event with `millis()` timestamps to the Serial Monitor and keep a simple paper "watering diary" for a week.